

EMSI — École Marocaine des Sciences de l'Ingénieur Module : Deep Learning
Année universitaire 2025–2026

RAPPORT DE PROJET

Guide de Référence pour l'Entraînement de Modèles de Deep Learning

MLP • CNN • RNN • LSTM • GRU • Architectures Hybrides

Réalisé par :	[NOM Prénom]
Encadré par :	Mme. HIDILA Zineb
Filière :	[Filière / Année]

Guide de Référence — Entraînement de Modèles Deep Learning

Table des matières

Table des matières	
2 1. Introduction	
4 1.1 Vue d'ensemble des architectures	
4 2. Préparation des données	
5 2.1 Exploration et compréhension du dataset	
5 2.2 Prétraitement	
5 2.2.1 Données tabulaires (MLP)	
5 2.2.2 Images (CNN)	
5 2.2.3 Séquences/Signaux (RNN/LSTM/GRU)	
6 2.3 Séparation des données	
6 3. Architectures en détail	7
3.1 Perceptron Multicouche (MLP)	
7 3.1.1 Structure type	
7 3.1.2 Points d'attention	
7 3.2 Réseaux de Neurones Convolutifs (CNN)	
8 3.2.1 Structure type (CNN personnalisé)	
8 3.2.2 Transfert d'apprentissage	
8 3.2.3 Augmentation de données	
8 3.3 Réseaux Récurrents (RNN, LSTM, GRU)	
9 3.3.1 Comparaison des variantes	
9 3.3.2 Configuration type (LSTM)	
9 3.4 Architectures hybrides	
10 3.4.1 CNN + LSTM	
10 3.4.2 CNN + MLP (Fusion multimodale)	
10 3.4.3 Stratégies de fusion	
10 4. Processus d'entraînement	

11 4.1 Fonctions de perte	11
11 4.2 Gestion du déséquilibre des classes	11
11 4.3 Optimiseurs	11
11 4.4 Schedulers de learning rate	11
11 4.5 Régularisation	11
12 5. Guide des hyperparamètres	12
13 5.1 Hyperparamètres clés par architecture	13
13 5.1.1 MLP	13
13 5.1.2 CNN	13
13 5.1.3 RNN / LSTM / GRU	13
13 5.2 Stratégies de recherche d'hyperparamètres	13
13	

Page 2

Guide de Référence — Entraînement de Modèles Deep Learning

6. Métriques d'évaluation	15
15 6.1 Métriques de classification	15
15 6.2 Visualisations obligatoires	15
15 6.3 Diagnostic par les courbes d'apprentissage	15
15 7. Interprétabilité des modèles	17
17 8. Bonnes pratiques et erreurs fréquentes	18
18 8.1 Checklist avant entraînement	18
18 8.2 Erreurs fréquentes	18
18 8.3 Structure type du code d'entraînement	18
18 9. Guide de rédaction du rapport	20
20 9.1 Structure attendue	20
20 9.2 Critères de qualité rédactionnelle	20
20 9.3 Ce qu'il faut éviter	20
20 10. Références bibliographiques recommandées	22
22 10.1 Articles fondateurs	22
22 10.2 Ressources pédagogiques	22
22	

Page 3

Guide de Référence — Entraînement de Modèles Deep Learning

1. Introduction

Ce guide de référence présente la méthodologie complète pour l'entraînement de modèles de deep learning, couvrant les architectures fondamentales (MLP, CNN, RNN, LSTM, GRU) ainsi que les architectures hybrides. Il constitue un cadre méthodologique que l'étudiant doit adapter à son propre projet.

L'objectif est de fournir une démarche structurée et reproductible, depuis la préparation des données jusqu'à l'analyse critique des

résultats, en passant par le choix d'architecture, l'optimisation des hyperparamètres et l'interprétation des modèles.

1.1 Vue d'ensemble des architectures

Architecture	Données cibles	Biais inductif	Cas d'usage typique
MLP	Tabulaires	Aucun	Classification/régression sur features structurées
CNN	Images, grilles 2D	Spatial (localité)	
RNN	Séquences courtes Séquences longues	Temporel (ordre)	NLP simple, séries courtes
LSTM		Temporel (mémoire longue)	Traduction, analyse de signaux
GRU	Séquences longues	Temporel (simplifié)	Alternative légère
Hybride	Spatio-temporel	Spatial + Temporel	Vidéo, LSTM, signaux multi-canaux
(CNN+LSTM)			

2. Préparation des données

2.1 Exploration et compréhension du dataset

Avant toute modélisation, une exploration approfondie du dataset est indispensable. Cette phase doit couvrir les points suivants :

- Description générale : nombre d'échantillons, nombre de variables/features, type de chaque variable (numérique, catégorielle, ordinale).
- Distribution de la variable cible : vérifier l'équilibre des classes et quantifier le ratio de déséquilibre.
- Statistiques descriptives : moyenne, médiane, écart-type, min/max pour les variables numériques.
- Valeurs manquantes : cartographier les taux de données manquantes par variable.
- Corrélations : matrice de corrélation pour identifier les redondances et les relations linéaires.
- Visualisations : histogrammes, boxplots, pairplots, heatmaps de corrélation.

Conseil : Toujours visualiser les données avant de coder le modèle. Un simple histogramme peut révéler un déséquilibre critique ou une distribution inattendue.

2.2 Prétraitement

2.2.1 Données tabulaires (MLP)

1. Gestion des valeurs manquantes : imputation par la médiane (numérique) ou le mode (catégorielle). Pour les taux supérieurs à 40%, envisager la suppression de la variable.
2. Encodage des variables catégorielles : One-Hot Encoding pour les variables nominales (10 catégories), Label Encoding pour les ordinales, Target Encoding si cardinalité élevée.
3. Normalisation/Standardisation : StandardScaler (z-score) ou MinMaxScaler selon la distribution. Appliquer après le split pour éviter le data leakage.
4. Sélection de features : suppression des variables à variance nulle, analyse de corrélation, feature importance préliminaire.

Attention : Toujours fitter le scaler sur le train set uniquement, puis transformer train, validation et test avec le même scaler. Fitter sur l'ensemble complet constitue un data leakage.

2.2.2 Images (CNN)

1. Redimensionnement : taille uniforme (ex. 224×224 ou 256×256). Choisir en fonction de la mémoire GPU disponible.

2. Normalisation : mise à l'échelle $[0, 1]$, puis normalisation par la moyenne et l'écart-type (ImageNet si transfert d'apprentissage : $\text{mean}=[0.485, 0.456, 0.406]$, $\text{std}=[0.229, 0.224, 0.225]$).
3. Augmentation de données (train uniquement) : rotations, retournements, recadrage aléatoire, ajustement de luminosité/contraste, ajout de bruit gaussien.

Page 5

Guide de Référence — Entraînement de Modèles Deep Learning

4. Conversion de format : DICOM vers PNG/JPG si nécessaire, gestion des canaux (niveaux de gris $\rightarrow$ 3 canaux pour les modèles pré-entraînés).

2.2.3 Séquences/Signaux (RNN/LSTM/GRU)

1. Rééchantillonnage : uniformiser la fréquence d'échantillonnage (ex. 500 Hz $\rightarrow$ 100 Hz pour réduire la complexité).
2. Segmentation : découpage en fenêtres de taille fixe (avec ou sans recouvrement).
 3. Normalisation : z-score par canal/variable ou normalisation min-max par séquence.
 4. Padding/Troncature : uniformiser la longueur des séquences (padding avec zéros ou troncature).
 5. Gestion du bruit : filtrage passe-bande si nécessaire, suppression des artefacts.

2.3 Séparation des données

Ensemble	Proportion	Rôle	Précautions
Train	70–	Entraînement du	
Validation	80%	modèle	Stratified
	10–	Tuning des	split
	15%	hyperparamètres	obli-
			ga-
			toire
			Jamais
			utilisé
			pour
			le gra-
			dient
Test	10–15%	Évaluation finale	Isolé jusqu'à
		unique	l'évaluation finale

Attention : Le test set ne doit être utilisé qu’une seule fois, pour l’évaluation finale. Toute utilisation répétée pour ajuster le modèle constitue un overfitting indirect.

Page 6

Guide de Référence — Entraînement de Modèles Deep Learning

3. Architectures en détail

3.1 Perceptron Multicouche (MLP)

Le MLP est l’architecture la plus directe pour les données tabulaires. Chaque neurone d’une couche est connecté à tous les neurones de la couche suivante (fully connected). La sortie d’un neurone est : $y = f(Wx + b)$.

3.1.1 Structure type

Couche	Type	Paramètres	Remarques
Entrée	Linear	$n_features \rightarrow 128$	
Cachée 1	Linear + ReLU + BN + Dropout	$128 \rightarrow 64$	Dimension = nom- bre de vari- ables BatchNorm avant ou après ReLU
Cachée 2	Linear + ReLU + BN +	$64 \rightarrow 32$	Dropout = 0.2 à 0.5
Sortie	Linear + Sig- moid/Softmax	$32 \rightarrow n_classes$	Sigmoid (bi- naire) / Soft- max

(multi-classe)

3.1.2 Points d'attention

- Architecture en entonnoir : réduire progressivement la dimension (ex. $256 \rightarrow 128 \rightarrow 64 \rightarrow 32$).
- Ne pas aller trop profond : 2 à 4 couches cachées suffisent généralement pour les données tabulaires.
- Batch Normalization : stabilise et accélère l'entraînement.
- Dropout : régularisation essentielle pour éviter le surapprentissage. Commencer à 0.3.

	Fonction de perte :
	BCEWithLogitsLoss
	(binaire) ou
	CrossEntropyLoss
•	(multi-classe).

Page 7

Guide de Référence — Entraînement de Modèles Deep Learning

3.2 Réseaux de Neurones Convolutifs (CNN)

Les CNN exploitent la structure spatiale des images grâce à la convolution (partage de poids, invariance par translation). Un bloc convolutif typique comprend : Conv2d $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool.

3.2.1 Structure type (CNN personnalisé)

Bloc	Opérations	Entrée $\rightarrow$ Sortie	Remarques
Bloc 1	Conv(3 $\times$ 3, 32) +	1 $\times$ 256 $\times$ 256 $\rightarrow$	Filtres larges en
Bloc 2	BN + ReLU +	32 $\times$ 128 $\times$ 128	début
	MaxPool(2)	32 $\times$ 128 $\times$ 128 $\rightarrow$	Doubler les filtres
	Conv(3 $\times$ 3, 64) +	64 $\times$ 64 $\times$ 64	
	BN + ReLU +		
	MaxPool(2)		

Bloc	Opérations	Entrée → Sortie	Remarques
Bloc 3		$64 \times 64 \times 64 \rightarrow$	Augmenter la
Bloc 4	Conv(3×3, 128) + BN + ReLU + Max- Pool(2) Conv(3×3, 256) + BN + ReLU + Adap- tiveAvg- Pool	$128 \times 32 \times 32 \rightarrow$ $128 \times 32 \times 32 \rightarrow$ $256 \times 1 \times 1$	profondeur Global pooling en sortie
Classif.	Linear(256, 128) + ReLU + Dropout + Lin- ear(128, 1)	$256 \rightarrow 1$	Couches denses finales

3.2.2 Transfert d'apprentissage

Le transfert d'apprentissage consiste à réutiliser un modèle pré-entraîné (ResNet, VGG, EfficientNet) et à adapter les dernières couches à la tâche cible. Deux stratégies principales :

- Feature extraction : geler toutes les couches convolutives, ne réentraîner que le classifieur. Rapide, efficace avec peu de données.
- Fine-tuning : dégeler progressivement les dernières couches convolutives. Plus performant mais nécessite plus de données et un learning rate faible ($1e-4$ à $1e-5$).

Conseil : Pour le fine-tuning, utiliser un learning rate différentiel : lr faible pour les couches pré-entraînées ($1e-5$), lr plus élevé pour le classifieur ($1e-3$).

3.2.3 Augmentation de données

Transformation	Paramètres typiques	Quand l'utiliser
RandomHorizontalFlip RandomRotation	p=0.5 degrees=15–30	Toujours (sauf si orientation est discrimi- nante) Images sans orientation fixe
GaussianBlur RandomAffine	kernel_size=3 translate=(0.1, 0.1)	Pour robustesse au bruit d'acquisition Pour simuler des décalages spatiaux

Page 8

Guide de Référence — Entraînement de Modèles Deep Learning

3.3 Réseaux Récurrents (RNN, LSTM, GRU)

Les architectures récurrentes traitent des données séquentielles en maintenant un état caché qui se propage dans le temps. Le RNN simple souffre de la disparition du gradient sur les longues séquences, ce qui a motivé le développement du LSTM et du GRU.

3.3.1 Comparaison des variantes

Critère	RNN simple	LSTM	GRU
État in- terne Portes	1 état caché (h) Au- cune	2 états (h + cellule c) 3 (en- trée, oubli, sor- tie)	1 état caché (h) 2 (reset, up- date)
Paramètres Mémoire longue	Le moins Faible	Le plus Excellente	Intermédiaire Très bonne

Critère	RNN simple	LSTM	GRU
Vitesse d'entraînement	Le plus rapide	Le plus lent	Intermédiaire
Quand l'utiliser	Prototypage rapide	Séquences longues, défaut	Compromis performance/vitesse

3.3.2 Configuration type (LSTM)

Paramètre	Valeur recommandée	Remarques
input_size	Nombre de features par timestep 64 à 256	Ex. 12 pour ECG 12 dérivations
hidden_size		
num_layers	1 à 3	Commencer à 128
bidirectional	True	2 couches = bon compromis
		Capture le contexte passé et futur
		Entre les couches LSTM (num_layers > 1)
dropout	0.2 à 0.5	
batch_first	True	Convention PyTorch : (batch, seq, features)

Note : Pour la classification, on utilise généralement le dernier état caché (ou la concaténation forward + backward si bidirectionnel) passé à une couche dense.

Page 9

Guide de Référence — Entraînement de Modèles Deep Learning

3.4 Architectures hybrides

Les architectures hybrides combinent les forces de plusieurs familles de réseaux pour traiter des données présentant des structures multiples (spatiale + temporelle, par exemple).

3.4.1 CNN + LSTM

Cette architecture utilise un CNN comme extracteur de features spatiales, puis un LSTM pour capturer les dépendances temporelles. Cas d'usage typiques : classification de vidéos, analyse de signaux multicanaux avec structure spatiale.

Pipeline : Entrée (batch, seq_len, C, H, W) $\rightarrow$ CNN par timestep $\rightarrow$ (batch, seq_len, n_features) $\rightarrow$ LSTM $\rightarrow$ (batch, hidden) $\rightarrow$ Dense $\rightarrow$ Prédiction.

3.4.2 CNN + MLP (Fusion multimodale)

Pour combiner des données hétérogènes (ex. image + données tabulaires), on extrait des embeddings de chaque branche puis on les concatène avant une couche de décision.

Pipeline : Branche image (CNN $\rightarrow$ embedding) + Branche tabulaire (MLP $\rightarrow$ embedding) $\rightarrow$ Concaténation $\rightarrow$ Dense $\rightarrow$ Prédiction.

3.4.3 Stratégies de fusion

Stratégie	Description	Avantages	Inconvénients
Early fusion	Concaténation des données brutes en entrée	Simple à implémenter	Dimensions in-
Late fusion	Concaténation des embeddings avant la décision	Flexible, modulaire	com-pati-bles Interactions limitées

Intermediate fusion	Fusion à un niveau intermédiaire du réseau	Interactions riches	Plus complexe à concevoir
Attention-based	Mécanisme d'attention cross-modal	Pondération adaptative	Coût computationnel élevé

Page 10

Guide de Référence — Entraînement de Modèles Deep Learning

4. Processus d'entraînement

4.1 Fonctions de perte

Tâche	Fonction de perte Py-Torch	Activation de sortie	Quand l'utiliser
Classification bi-naire	nn.BCEWithLogitsLoss()	Aucune	2 classes, label unique
Classification multi-classe	nn.CrossEntropyLoss()	Aucune (logits)	N classes mutuellement exclusives
Classification multi-labels	nn.BCEWithLogitsLoss()	Aucune (logits)	Plusieurs labels simultanés
Régression	nn.MSELoss() ou nn.L1Loss()	Aucune (linéaire)	Prédiction de valeurs continues

Attention : BCEWithLogitsLoss attend des logits (pas de sigmoïde en sortie).

CrossEntropyLoss attend des logits (pas de softmax). Appliquer sigmoïde/softmax avant ces loss fonctions est une erreur fréquente.

4.2 Gestion du déséquilibre des classes

Le déséquilibre des classes est un problème récurrent en deep learning appliqué. Plusieurs stratégies peuvent être combinées :

- Pondération de la loss : calculer les poids inversement proportionnels à la fréquence de chaque classe (paramètre weight de CrossEntropyLoss ou pos_weight de BCEWithLogitsLoss).
- Suréchantillonnage de la classe minoritaire : SMOTE (données tabulaires), augmentation ciblée (images).
- Sous-échantillonnage de la classe majoritaire : à utiliser avec précaution (perte d'information).
- Focal Loss : variante de la BCE qui réduit le poids des exemples faciles. Utile pour les déséquilibres sévères.

4.3 Optimiseurs

Optimiseur	Learning rate typique	Avantages	Cas d'usage
SGD + Mo- men- tum Adam	0.01 à 0.1 1e-3 à 1e-4	Bonne généralisation Convergence rapide, adaptatif	CNN avec trans- fert, en- traîne- ment long Défaut pour la plu- part des cas

Optimiseur	Learning rate typique	Avantages	Cas d'usage
AdamW RMSProp	1e-3 à 1e-4 1e-3	Adam + weight decay décou- plé Bon pour les RNN	Fine-tuning de modèles pré-entraînés Séquences, gradi- ents insta- bles

4.4 Schedulers de learning rate

Page 11

Guide de Référence — Entraînement de Modèles Deep Learning

Le scheduler ajuste le learning rate au cours de l'entraînement pour améliorer la

convergence :

- StepLR : réduit le lr d'un facteur gamma tous les step_size époques. Simple et

efficace.

- ReduceLROnPlateau : réduit le lr quand la métrique de validation stagne.

Recommandé comme défaut.

- CosineAnnealingLR : décroissance en cosinus. Populaire pour le fine-tuning.
- OneCycleLR : monte puis descend le lr en un cycle. Peut accélérer la convergence.

4.5 Régularisation

Technique	Où l'appliquer	Paramètres	Effet
Dropout Batch Normal- iza- tion	Après les couches denses/LSTM Après Conv/Linear, avant ou après ReLU	$p = 0.2$ à 0.5 momentum=0.1	Désactive des neu- rones aléa- toire- ment Normalise les ac- tiva- tions par batch
Weight Decay (L2) Early Stop- ping Data Aug- men- tation Label Smooth- ing	Via l'optimiseur Boucle d'entraînement Pipeline de données (train) Fonction de perte	$1e-4$ à $1e-2$ patience $= 5$ à 15 Variable $= 0.1$	Pénalise les grands poids Arrête quand val_loss stagne Augmente la diver- sité virtuelle Adoucit les cibles (moins de confi- ance)

Page 12

Guide de Référence — Entraînement de Modèles Deep Learning

5. Guide des hyperparamètres

5.1 Hyperparamètres clés par architecture

5.1.1 MLP

Hyperparammètre	Plage de recherche	Défaut recommandé
Nombre de couches cachées	2 à 4	3
Neurones par couche	32 à 512	128, 64, 32 (entonnoir)
Learning rate	1e-4 à 1e-2	1e-3 (Adam)
Batch size	32 à 256	64
Dropout	0.1 à 0.5	0.3
Époques	50 à 200	100 + early stopping
Weight decay	0 à 1e-2	1e-4

5.1.2 CNN

Hyperparammètre	Plage de recherche	Défaut recommandé
Nombre de blocs convolutifs	3 à 6	4
Filtres par bloc	16 à 512	32, 64, 128, 256 (doublement)
Taille du noyau	3×3, 5×5, 7×7	3×3 (standard)
Learning rate (from scratch)	1e-3 à 1e-2	1e-3
Époques	20 à 100	50 + early stopping
Augmentation	Légère à agressive	Modérée

5.1.3 RNN / LSTM / GRU

Hyperparammètre	Plage de recherche	Défaut recommandé
Bidirectionnel	True / False	True
Dropout inter-couches	0.1 à 0.5	0.3

Hyperparammètre	Plage de recherche	Défaut recommandé
Learning rate	1e-4 à 1e-2	1e-3
Gradient clipping	0.5 à 5.0	1.0
Longueur de séquence	Dépend du signal	1000 (après rééchantillonnage)

5.2 Stratégies de recherche d'hyperparammètres

- Grid Search : exhaustif mais coûteux. Réserver pour 2–3 hyperparammètres avec peu de valeurs.
- Random Search : plus efficace que le grid search pour des espaces larges. 50–100 itérations.

Page 13

Guide de Référence — Entraînement de Modèles Deep Learning

•	Recherche manuelle guidée : commencer par les défauts, varier un paramètre à la
---	---

fois. Approche pragmatique recommandée pour les projets étudiants.

Conseil : Commencer par le learning rate (le plus impactant), puis batch size, puis profondeur, puis régularisation. Toujours fixer un seed pour la reproductibilité.

Page 14

Guide de Référence — Entraînement de Modèles Deep Learning

6. Métriques d'évaluation

6.1 Métriques de classification

Métrique	Formule / Description	Quand l'utiliser
Accuracy	$TP + TN / \text{Total}$	Classes équilibrées
Précision	$TP / (TP + FP)$	uniquement Coût élevé des faux positifs

Métrique	Formule / Description	Quand l'utiliser
Rappel (Sensibilité) F1-Score	$TP / (TP + FN)$ $2 \times (Précision \times Rappel) / (Précision + Rappel)$	Coût élevé des faux négatifs (médical)
AUC-ROC Matrice de confusion	Aire sous la courbe ROC Tableau TP/TN/FP/FN	Compromis précision/rappel Métrique globale de discrimination
Macro F1 Spécificité	Moyenne des F1 par classe $TN / (TN + FP)$	Analyse détaillée des erreurs Classes multiples déséquilibrées Taux de vrais négatifs

Attention : En médecine, le rappel (sensibilité) est souvent plus critique que la précision. Un faux négatif (maladie non détectée) est généralement plus dangereux qu'un faux positif (examen supplémentaire inutile).

6.2 Visualisations obligatoires

Chaque expérience doit produire les visualisations suivantes :

1. Courbes d'apprentissage : loss et métrique principale (accuracy, F1) sur train et validation en fonction des époques. Permet de diagnostiquer l'overfitting/underfitting.
2. Matrice de confusion : sur le test set, normalisée ou non. Identifie les confusions inter-classes.
3. Courbe ROC et AUC : pour chaque classe en classification binaire/multi-classe.
4. Tableau récapitulatif : toutes les configurations testées avec leurs hyperparamètres et métriques.

6.3 Diagnostic par les courbes d'apprentissage

Observation	Diagnostic	Action corrective
train_loss	Overfitting	Augmenter
↓↓, val_loss	Underfitting	dropout,
↓ puis ↑		réduire
train_loss ↓		modèle, early
lent, val_loss		stopping
↓ lent		Augmenter le
		modèle,
		réduire régu-
		larisation,
		augmenter lr
train_loss et	Modèle trop	Augmenter capacité ou
val_loss	simple ou lr	lr
stagnent	trop faible	Continuer, surveiller la
haut	Bon entraînement	divergence
train_loss et		
val_loss ↓		
ensemble		
val_loss	Learning	Réduire lr, augmenter
oscille	rate trop	batch size
fortement	élevé ou	
	batch trop	
	petit	

Page 15

Guide de Référence — Entraînement de Modèles Deep Learning

Page 16

Guide de Référence — Entraînement de Modèles Deep Learning

7. Interprétabilité des modèles

L'interprétabilité est un enjeu majeur en deep learning appliqué, particulièrement dans les

domaines critiques (santé, finance, sécurité). Un modèle performant mais opaque est

difficilement adoptable en pratique clinique.

Architecture	Méthode d'interprétation	Ce qu'elle révèle
CNN	Grad-CAM	Quelles régions spatiales activent la décision
CNN	Saliency	Gradients de la sortie par rapport aux pixels d'entrée
	Maps	Quels segments du signal sont discriminants
RNN/LSTM	Gradient par rapport à l'entrée	Visualisation de l'espace latent appris
Tous	t-SNE / UMAP des embeddings	

Conseil : Inclure au moins une méthode d'interprétation par architecture dans le rapport.

Discuter la cohérence des résultats avec les connaissances du domaine.

Page 17

Guide de Référence — Entraînement de Modèles Deep Learning

8. Bonnes pratiques et erreurs fréquentes

8.1 Checklist avant entraînement

1. Seed fixé (torch.manual_seed, np.random.seed, random.seed) pour la reproductibilité.
2. Data leakage vérifié : aucune fuite d'information du test vers le train.
3. Dimensions vérifiées : input shape, output shape, nombre de classes.
4. Baseline établie : modèle simple (régression logistique, prédiction majoritaire) comme référence.
5. GPU disponible : torch.cuda.is_available(), device = torch.device('cuda' if available).
6. DataLoader configuré : shuffle=True pour train, shuffle=False pour val/test, num_workers adapté.

8.2 Erreurs fréquentes

Erreur	Conséquence	Solution
Oublier <code>model.eval()</code> en évaluation Softmax avant CrossEntropyLoss	Dropout et BN actifs → métriques faussées Double application → convergence dégradée	Toujours <code>model.eval()</code> + <code>torch.no_grad()</code> CrossEntropyLoss attend des logits bruts
Normaliser sur tout le dataset Ignorer le déséquilibre des classes Batch size trop grand Pas de gradient clipping (RNN)	Data leakage → métriques optimistes Modèle biaisé vers la classe majoritaire Mauvaise généralisation Explosion du gradient → NaN	Fitter le scaler sur train uniquement Pondération de la loss, SMOTE, augmentation 32 à 64 par défaut <code>torch.nn.utils.clip_grad_norm_(max_norm=1.0)</code>
Oublier <code>optimizer.zero_grad()</code> Pas d'early stopping	Accumulation de gradients → instabilité Surapprentissage non détecté	Appeler en début de chaque itération Monitorer <code>val_loss</code> , <code>patience=5-15</code>

8.3 Structure type du code d'entraînement

Chaque notebook ou script d'entraînement doit suivre cette structure logique :

1. Imports et configuration (device, seeds, hyperparamètres).
2. Chargement et exploration des données.
3. Prétraitement et création des DataLoaders.
4. Définition du modèle (classe héritant de `nn.Module`).
5. Définition de la loss, de l'optimiseur et du scheduler.
6. Boucle d'entraînement avec logging (`train_loss`, `val_loss`, métriques par époque).

7. Évaluation sur le test set (une seule fois, après sélection du meilleur modèle).
8. Visualisations (courbes, matrices, Grad-CAM, etc.).
9. Analyse critique et conclusions.

Page 18

Guide de Référence — Entraînement de Modèles Deep Learning

Page 19

Guide de Référence — Entraînement de Modèles Deep Learning

9. Guide de rédaction du rapport

Le rapport constitue le livrable principal du projet. Il doit démontrer la compréhension

théorique, la rigueur expérimentale et la capacité d'analyse critique de l'étudiant.

9.1 Structure attendue

Section	Contenu attendu	Pages estimées
Page de garde	Titre, auteur, encadrant,	1
Table des matières	filère, année Générée automatiquement	1
Introduction	Contexte,	1-2
Cadre théorique	problématique, objectifs, plan Fondements mathématiques de chaque architecture	3-5
Partie expérimentale	Dataset, pré-traitement,	8-15 (par partie)
Discussion transversale	architecture, résultats, analyse Comparaison architectures, adéquation données-modèle	2-3

Section	Contenu attendu	Pages estimées
Impact sociétal / Éthique Conclusion	Implications, biais, limites, contexte local Synthèse, résultats clés, perspectives	1–2 1
Références Annexes	Articles, datasets, frameworks (format IEEE/APA) Code clé, configura- tions, résultats complémen- taires	1–2 Variable

9.2 Critères de qualité rédactionnelle

- Précision : chaque affirmation technique doit être justifiée (formule, référence, résultat expérimental).
- Clarté : éviter les phrases vagues ("le modèle fonctionne bien"). Quantifier systématiquement.
- Esprit critique : ne pas se limiter à rapporter les résultats. Analyser les échecs, discuter les limites, proposer des améliorations.
- Cohérence : le fil conducteur (problématique) doit être visible tout au long du rapport.
- Références : citer les articles fondateurs des architectures utilisées (LeCun, Hochreiter, He, etc.).

9.3 Ce qu'il faut éviter

- Copier-coller du code sans explication. Le code doit être commenté et justifié dans le texte.

- Présenter des résultats sans analyse. Un tableau de métriques seul ne constitue pas une analyse.
- Ignorer les mauvais résultats. Un modèle qui échoue est aussi instructif qu'un modèle performant, à condition d'analyser pourquoi.

•	Utiliser l'accuracy comme unique métrique sur des classes déséquilibrées.
---	---

Page 20

Guide de Référence — Entraînement de Modèles Deep Learning

•	Omettre les détails de reproductibilité : seeds, versions de librairies,
---	--

hyperparamètres.

Page 21

Guide de Référence — Entraînement de Modèles Deep Learning

10. Références bibliographiques recommandées

10.1 Articles fondateurs

- [1] Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [4] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.

- [6] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
- [7] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015.
- [8] Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 15, 1929–1958.
- [9] Ioffe, S. & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML 2015.
- [10] Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.
- [11] Lundberg, S. & Lee, S. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.

10.2 Ressources pédagogiques

- [12] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [13] PyTorch Documentation officielle : <https://pytorch.org/docs/stable/>
- [14] PyTorch Tutorials : <https://pytorch.org/tutorials/>
- [15] Scikit-learn : <https://scikit-learn.org/stable/>